Name: _____ Cohort/Batch: _____ Date: _____ Score: _____

VISUAL STUDIO CODE PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 conceptual Q&A Developer Tools

TOTAL: 10 QUESTIONS

Q1 What is Visual Studio Code and what problem in Developer Tools does it solve? — Threat Model

Q2 What are the core components or concepts in Visual Studio Code? — Architecture

Q3 How does Visual Studio Code compare to its main alternatives? — Configuration

Q4 How do you get started with Visual Studio Code for a new project? — Hardening

Q5 What configuration options most affect Visual Studio Code's behavior in production? — SSO / IdP

Q6 How does Visual Studio Code handle reliability and high availability? — Observability

Q7 What observability does Visual Studio Code provide out of the box? — Lifecycle

Q8 What are common performance bottlenecks in Visual Studio Code? — Compliance

Q9 What security best practices apply when running Visual Studio Code? — Testing

Q10 What mistakes do teams commonly make when adopting Visual Studio Code? — Tuning

ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1 Visual Studio Code is a tool

Mitigation

Visual Studio Code is a tool or platform that automates or simplifies a specific concern in the Developer Tools domain, reducing manual effort and operational complexity for engineering teams.

A2 Visual Studio Code has key building

Primitives

Visual Studio Code has key building blocks that work together: a configuration layer defines intent, a runtime layer enforces it, and an API or CLI layer allows operators to manage and observe the system.

A3 Visual Studio Code trades off simplicity

Hardening

Visual Studio Code trades off simplicity against feature richness differently than its alternatives. Choose Visual Studio Code when its specific strengths performance, ecosystem, or ease of use align with your team's primary constraints.

A4 Install Visual Studio Code via its

Gotchas

Install Visual Studio Code via its package manager or binary release. Follow the quickstart to initialize a project, configure the minimal required settings, and run the default command to verify the setup works end-to-end.

A5 Production-critical settings typically include

Federation

Production-critical settings typically include concurrency limits, timeout values, retry policies, resource limits, and logging levels. Review the official production hardening guide before deploying.

A6 Visual Studio Code provides HA through

Observability

Visual Studio Code provides HA through leader election, replication, or stateless horizontal scaling. Deploy at least two instances across availability zones and configure health checks for automatic failover.

A7 Visual Studio Code exposes metrics

Automation

Visual Studio Code exposes metrics (Prometheus endpoint or built-in dashboard), structured logs, and optionally distributed traces. Define SLOs on the key metrics and alert before users are affected.

A8 Performance bottlenecks typically stem from

Governance

Performance bottlenecks typically stem from misconfigured connection pools, large payload sizes, insufficient worker threads, or missing caches. Profile with built-in diagnostics before optimizing.

A9 Run Visual Studio Code with the

Assessment

Run Visual Studio Code with the principle of least privilege, enable TLS for all communication, use a secrets manager for credentials, and keep the software patched to the latest stable release.

A10 Common pitfalls include skipping the

Optimization

Common pitfalls include skipping the documentation and learning the API by trial and error, using default configurations in production, lacking a runbook for operational issues, and not testing failover scenarios.